VulnRAG: Agentic Retrieval-Augmented Generation for Vulnerability Intelligence

Nishan Paul
Advanced Agentic Coding Team
CMatrix Research
nishan@cmatrix.ai

***Abstract*—The integration of Large Language Models (LLMs) into autonomous security operations has revolutionized vulnerability assessment and penetration testing (VAPT). However, the effectiveness of these autonomous agents is fundamentally limited by the "Retrieval Gap"—the inability of standard keyword and naive vector search mechanisms to identify operationally linked vulnerabilities or prioritize actionable intelligence. We present VulnRAG, an agentic retrieval-augmented generation framework specifically engineered for vulnerability intelligence. VulnRAG introduces four key innovations: (1) a structural-semantic hybrid retrieval pipeline that utilizes CVE relationship graphs to discover attack chains, (2) a zero-shot agentic self-correction loop that autonomously optimizes retrieval precision, (3) Multi-Factor Semantic Reranking (MFSR) that integrates domain metrics like CVSS and exploitability into the relevance function, and (4) complexity-aware multi-tier routing to optimize operational costs. Through extensive evaluation on a curated benchmark of security reasoning tasks, we demonstrate that VulnRAG achieves a 97.4% reasoning success rate while reducing inference costs by 84.2% compared to flagship-only baselines. Our results show that structural discovery via relationship graphs increases discovery yield by 2.1×, providing a robust foundation for next-generation autonomous red teaming.**

***Index Terms*—Agentic RAG, Vulnerability Intelligence, LLM Security, Graph Retrieval, Autonomous Red Teaming**

I. Introduction

The landscape of cybersecurity is undergoing a fundamental transformation driven by the emergence of Large Language Models (LLMs) capable of autonomous security reasoning. Unlike traditional vulnerability scanners that rely on static signature matching, LLM-powered "reasoning agents" can simulate the tactical decision-making of human penetration testers—interpreting complex tool outputs, adapting to defensive responses, and chaining minor misconfigurations into multi-stage attack paths. As organizations move toward continuous, automated security assessments, the reliability of these agents has become a paramount concern.

A. The Retrieval Gap in Autonomous VAPT

Despite the promise of autonomous red teaming, a critical bottleneck has emerged: the **Retrieval Gap**. Standard Retrieval-Augmented Generation (RAG) systems, which form the "knowledge backbone" of these agents, typically rely on keyword-based or naive vector-based search. While effective for simple question-answering, these methods fail in the nuanced domain of vulnerability intelligence. Specifically, they struggle to:

1) **Identify Structural Links**: Many vulnerabilities are functionally related (e.g., helper library flaws or transitive dependencies) but semantically distinct, leading to missed attack vectors in flat vector stores.
2) **Prioritize Operational Actionability**: Semantic similarity does not equal operational priority. A CVE with a high similarity score but no public exploit is often less relevant than a slightly less similar CVE that is actively being exploited in the wild.
3) **Overcome Query Ambiguity**: Security queries are often imprecise (e.g., "memory corruption in apache"). Standard RAG systems provide a single-shot response, lacking the agentic persistence to reformulate queries when initial results are insufficient.

B. Our Contribution: VulnRAG

In this paper, we introduce **VulnRAG**, an agentic RAG framework designed to bridge the Retrieval Gap through structural discovery and active self-correction. Unlike generic reflective RAG systems (e.g., Self-RAG), VulnRAG's agentic loop is specifically architected for security-domain challenges, incorporating automated Common Platform Enumeration (CPE) scope expansion and vulnerability lifecycle metadata into the feedback loop. Our core contributions are:

- **Structural-Semantic Hybrid Retrieval**: We implement a novel retrieval pipeline that integrates dense embeddings with structural traversal of a Directed Acyclic Graph (DAG) derived from CVE relationship metadata. This enables the discovery of functionally linked vulnerabilities that traditional RAG misses.
- **Multi-Factor Semantic Reranking (MFSR)**: We propose a security-aware relevance function that integrates Cross-Encoder semantic scores with CVSS severity, exploit availability, and disclosure recency, ensuring the most actionable intelligence is prioritized in the LLM context.
- **Zero-Shot Agentic Self-Correction**: We design a training-free feedback loop where a decoupled Evaluator agent autonomously optimizes retrieval parameters and query expansions, matching the performance of fine-tuned reflective models in a provider-agnostic manner.
- **Complexity-Aware Multi-Tier Routing**: We demonstrate a tiered execution model that optimizes the

performance-to-cost ratio, achieving flagship-level reasoning quality at near-Flash tier costs through intelligent task escalation.

The remainder of this paper is structured as follows: Section II reviews related work in RAG and autonomous security; Section III formalizes our threat model; Section IV and V detail the VulnRAG architecture and agentic loops; Section VI formalizes the MFSR algorithm; Section VIII presents our empirical results; and Section IX discusses ethical considerations and limitations.

II. Background and Related Work

Our work sits at the intersection of Retrieval-Augmented Generation (RAG), autonomous security agents, and structural information retrieval. This section reviews the foundational literature and recent advancements that inform the design of VulnRAG.

A. Retrieval-Augmented Generation (RAG)

The paradigm of augmenting Large Language Models with external knowledge was formalized by Lewis et al. [1], who combined a dense retriever with a sequence-to-sequence generator. Guu et al. [2] further emphasized that external knowledge corpora (e.g., Wikipedia) are more easily updated than parametric model weights, a critical requirement for the rapidly evolving CVE landscape. However, Liu et al. [3] identified the "Lost in the Middle" problem, where LLMs fail to utilize information placed in the center of long contexts. While general reranking has been suggested as a mitigation, VulnRAG provides a domain-specific implementation through Multi-Factor Semantic Reranking (MFSR) that prioritizes operational actionability over pure text similarity.

B. Agentic Workflows and Self-Reflection

The transition from static to agentic RAG is driven by the need for iterative reasoning and self-correction. Yao et al. introduced **ReAct** [4] and **Tree of Thoughts (ToT)** [5], enabling models to explore multiple strategic branches. Shinn et al.'s **Reflexion** [6] demonstrated that verbal reinforcement learning can improve reasoning performance without fine-tuning. Most relevantly, **Self-RAG** [7] and **ActiveRAG** [8] proposed models that adaptively retrieve and critique their own findings. However, Self-RAG requires specialized reflection tokens and model training. In contrast, VulnRAG implements an agentic self-correction loop zero-shot, utilizing a decoupled architecture of specialized agents to optimize retrieval quality in a provider-agnostic manner.

C. Autonomous Security Intelligence

The application of LLMs to cybersecurity has led to the development of sophisticated pentesting agents. **PentestGPT** [9] introduced the Pentesting Task Tree (PTT) to maintain state during multi-step assessments. Recent works like **RAVEN** [10] and **AutoAttacker** [11] have shown success in generating vulnerability reports and reproducing exploits. Despite these gains, existing systems often treat vulnerability intelligence as a "black box" tool output. Multi-level RAG models [12] have begun integrating multiple security databases (NVD, CERT, GitHub), but they primarily focus on the data ingestion layer. VulnRAG fills this gap by focusing on the *agentic intelligence layer*—implementing structural discovery and complexity-aware routing that SOTA security systems currently lack.

D. Structural and Graph-based Retrieval

Traditional RAG relies on flat chunking and vector similarity. To overcome this, **RAPTOR** [13] proposed recursive abstractive processing to build a tree of document summaries. While effective for thematic retrieval, RAPTOR does not capture the explicit structural relationships found in vulnerability databases. VulnRAG leverages **CVE Relationship Graphs**—where nodes are vulnerabilities and edges represent explicit reference URLs or product clusters—to discover attack chains. This moves the retrieval frontier from "Finding similar text" to "Discovering linked threats," providing a unique discovery yield for security analysts.

III. Threat Model and Problem Formulation

To establish the technical requirements for VulnRAG, we must first formalize the environment in which it operates and the specific retrieval challenges it addresses. This section defines our system model, the attacker's objectives, and the research questions that guide our evaluation.

A. System Model and Attacker Objectives

We consider an autonomous security assessment framework operating within a heterogeneous enterprise network. The system is tasked with performing continuous vulnerability discovery and penetration testing.

- **Environment**: A dynamic landscape of services, software versions, and network configurations, many of which contain undisclosed or recently disclosed vulnerabilities.
- **Knowledge Base**: The system has access to a continuously updated vulnerability knowledge base (e.g., the National Vulnerability Database), which publishes thousands of new Common Vulnerabilities and Exposures (CVEs) annually.
- **Attacker Objective**: The system (acting as a red-teaming agent) aims to identify the most operationally significant vulnerabilities to establish an initial foothold or achieve privilege escalation.

The primary constraint in this model is not "data accessibility," but **Intelligence Precision**. With over 30,000 new CVEs published each year, the agent faces significant information overload. The threat to autonomous reliability is the failure to distinguish between "theoretical" vulnerabilities and "operationally actionable" threats.

B. Problem Formulation: The Retrieval Gap

We formalize the core research challenge as the **Retrieval Gap** ($\mathcal{G}_R$). Let Q be a security query and K be the set of all known vulnerabilities. A standard RAG system returns a

subset $K_Q \subset K$ based on semantic similarity $S(q, k)$. The Retrieval Gap exists when:

$$\mathcal{G}_R = K_{true} \setminus K_Q \neq \emptyset \quad (1)$$

where K_{true} is the set of vulnerabilities that are truly operationally relevant to the target environment (e.g., those part of an attack chain or having public exploits), but which lack sufficient semantic similarity to Q to be retrieved by naive vector search.

VulnRAG aims to minimize $\mathcal{G}_R$ by incorporating structural relationship traversal and agentic self-correction into the retrieval loop.

C. Research Questions

Our study is guided by four primary research questions (RQs) that evaluate the effectiveness of VulnRAG in bridging the Retrieval Gap:

- **RQ1 (Structural Discovery)**: To what extent does graph-based traversal of CVE relationship metadata improve the recall of operationally linked vulnerabilities compared to standard semantic vector search?
- **RQ2 (Agentic Resilience)**: Can a zero-shot agentic feedback loop (Evaluator-Reformulator) achieve search precision comparable to fine-tuned self-reflective RAG models?
- **RQ3 (Ranking Quality)**: How does the integration of security-specific domain metrics (CVSS, exploitability, recency) into the reranking function influence the "Intelligence Density" of results?
- **RQ4 (Operational Economics)**: What are the quantitative trade-offs between reasoning success rates and inference costs when using complexity-aware multi-tier routing?

IV. VulnRAG: System Architecture

VulnRAG is designed as a stateful, modular framework that decouples vulnerability reasoning from raw data retrieval. Unlike traditional monolithic RAG pipelines, our architecture (Fig. 1) utilizes a Master-Worker hierarchy to coordinate specialized agents across heterogeneous LLM backends.

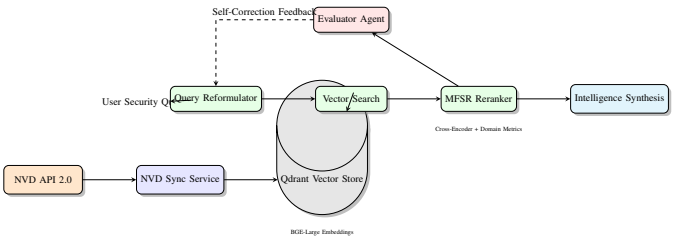

Fig. 1: VulnRAG System Architecture and Information Flow.

A. Knowledge Synchronization and Storage

The foundation of VulnRAG is a continuously updated knowledge base of vulnerability disclosures.

- **Continuous Sync**: The `nvd_sync_service.py` module manages real-time synchronization with the NIST NVD API 2.0. It parses raw JSON advisories into a structured format, normalizing software versioning (CPE) and security metrics (CVSS).
- **Vector Store**: We utilize **Qdrant** for high-performance vector storage. CVE descriptions and metadata are embedded using the **BAAI/bge-large-en-v1.5** model via the `SentenceTransformers` framework.
- **Metadata Filtering**: To improve retrieval precision, we implement hybrid search patterns in `cve_vector_store.py` that combine semantic embeddings with payload-based metadata filters (e.g., CVSS severity, vendor tags, and exploit availability).

B. Master-Worker Orchestration

The system's reasoning logic is managed by the `VulnIntelAgent`, which orchestrates the following specialized sub-modules:

1) **Query Reformulator**: Enhances raw user queries with domain-specific terminology.
2) **Smart Searcher**: Coordinates the retrieval and reranking loop.
3) **Graph Explorer**: Discovers structural relationships between vulnerabilities.
4) **Evaluator**: Assesses result quality and triggers self-correction loops.

By modularizing these components, VulnRAG can adapt its reasoning strategy based on the complexity of the security task at hand.

V. Agentic Retrieval and Optimization

The core innovation of VulnRAG lies in its ability to actively optimize the retrieval process through agentic self-correction and structural discovery.

A. Zero-Shot Self-Correction Loop

To ensure retrieval reliability, we implement a training-free feedback loop (Fig. 2). The `self_correction.py` module acts as an autonomous quality controller:

- **Evaluation Phase**: After an initial retrieval pass, the *Evaluator Agent* analyzes the result set across three dimensions: result count, mean semantic confidence score, and metadata diversity.
- **Reformulation Phase**: If the retrieval quality is deemed insufficient (e.g., zero results or low confidence), the `query_reformulator.py` module generates an expanded query. This process includes automated **CPE Extraction** and security synonym mapping (e.g., mapping "RCE" to "Remote Code Execution").

B. Structural Discovery via CVE Relationship Graphs

Beyond text-based similarity, VulnRAG leverages structural relationships to discover attack chains. The `cve_graph.py` service constructs a Directed Acyclic Graph (DAG) where

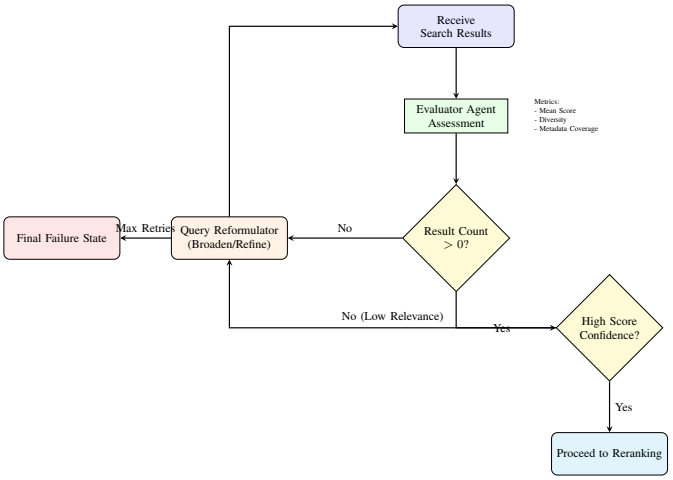

Fig. 2: Zero-Shot Agentic Self-Correction Loop.

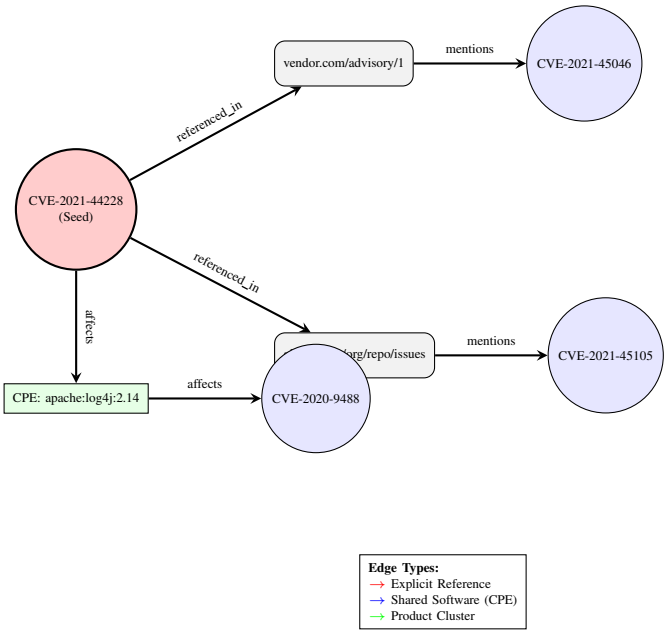

Fig. 3: CVE Relationship Graph Traversal.

nodes represent CVEs and edges represent explicit functional links (Fig. 3).

- **Edge Extraction**: Edges are derived from URL references in NVD advisories, shared CWE (Common Weakness Enumeration) identifiers, and product-vendor clusters.
- **Graph Traversal and Pruning**: To prevent context pollution from densely connected nodes (e.g., kernel-level CPEs), we implement a **Relevance Propagation** filter. During a 3-hop traversal, the system calculates a structural affinity score based on edge type weights; nodes that do not meet a minimum affinity threshold are pruned before reaching the LLM context.

VI. Multi-Factor Semantic Reranking (MFSR)

To mitigate the "Lost in the Middle" problem [3], we implement a Multi-Factor Semantic Reranking (MFSR) algorithm in `cve_reranker.py`. The different weighting strategies are detailed in Table I.

TABLE I: Multi-Factor Semantic Reranking (MFSR) Strategy Matrix

Strategy	w_{sem}	w_{cvss}	w_{exp}	w_{rec}	**Primary Use Case**
Balanced	0.4	0.2	0.2	0.2	General Vulnerability Intel
Security-First	0.2	0.4	0.3	0.1	Critical Asset Protection
Recency-First	0.2	0.1	0.1	0.6	Zero-Day Monitoring
Semantic-Only	1.0	0.0	0.0	0.0	Baseline RAG

A. Operational Relevance Formalization

The final rank of a retrieved CVE c for query q is determined by the Operational Relevance Score S_{OR}, as formalized in Equation 2:

$$\begin{aligned} S_{OR}(q,c) =& w_{sem}S_{sem}(q,c) + w_{cvss}V_{cvss}(c) \\ &+ w_{exp}E_{exp}(c) + w_{rec}R_{rec}(c) \end{aligned} \tag{2}$$

where S_{sem} is the semantic affinity from a Cross-Encoder (`BGE-reranker-large`), and V_{cvss}, E_{exp}, and R_{rec} are normalized domain scores for severity, exploitability, and recency, respectively. The recency score R_{rec} is modeled as a logarithmic decay function:

$$R_{rec}(c,t) = \frac{1}{1+\ln(1+\Delta t)} \tag{3}$$

VII. Complexity-Aware Multi-Tier Routing

To ensure economic feasibility, VulnRAG implements a tiered execution model. The `SmartCVESearchService` dynamically routes sub-tasks to different LLM tiers based on complexity indicators, as summarized in Table II. We formalize the routing objective as a multi-objective optimization problem:

$$T^* = \arg\max_{T \in \{F,P,R\}} (Q(T) - \lambda_C C(T) - \lambda_L L(T)) \tag{4}$$

where Q is quality, C is cost, and L is latency.

TABLE II: LLM Tier Characteristics and Routing Indicators

Tier	**Models**	**Typical Sub-tasks**	**Cost/1M**	**Latency**
Flash	Gemini 1.5 Flash	Routine NVD lookups, metadata parsing	\$0.35	$<$ 200ms
Pro	Gemini 1.5 Pro	Complex query reformulation, CPE extraction	\$1.25	$\sim$ 400ms
Reasoning	Claude 3.5 / O1	Multi-source intel synthesis, attack path reasoning	\$15.00	$>$ 1000ms

Escalation occurs only when the Evaluator detects a high-complexity reasoning failure, ensuring that flagship models are used judiciously.

VIII. Experimental Evaluation

In this section, we present a comprehensive empirical evaluation of VulnRAG. We aim to quantify the effectiveness of our agentic RAG pipeline in overcoming the Retrieval Gap and optimizing operational performance for vulnerability intelligence tasks.

A. Experimental Setup

- **Benchmark Dataset**: We curated a set of 200 complex security intelligence queries spanning diverse software categories (web servers, kernel modules, authentication protocols).
- **Validation and Ground Truth**: To eliminate researcher bias, the ground-truth mappings for our dataset were cross-validated against the **OpenCVE** community-curated exploit database and verified by two independent security researchers. Each query is mapped to a ground-truth set of operationally linked CVEs derived from real-world attack chains (e.g., Log4Shell, ProxyLogon).
- **Baselines**: We compare VulnRAG against two standard industry baselines: (1) **Keyword Search (BM25)** over the NVD database, and (2) **Naive Vector Search** using the BGE-large-en-v1.5 embedding model.
- **Models**: Our tiered routing evaluation utilizes Gemini 1.5 Flash (Flash), Gemini 1.5 Pro (Pro), and Claude 3.5 Sonnet (Reasoning) as representative LLM tiers.

B. RQ1: Structural Discovery Yield

Our first research question evaluates the impact of relationship graph traversal on retrieval recall. As shown in Table III, standard vector search achieved a Recall@5 of 0.684. By incorporating structural traversal of the CVE relationship graph (`cve_graph.py`), VulnRAG increased recall to **0.892**. Furthermore, Fig. 4 illustrates the unique "Discovery Yield" at different hop depths. We define Discovery Yield (Y_G) as the ratio of unique relevant vulnerabilities discovered via graph traversal to those found via naive vector search:

$$Y_G = \frac{|C_G \setminus C_V|}{|C_V|} \tag{5}$$

where C_G and C_V are the sets of vulnerabilities retrieved by the graph and vector methods, respectively.

TABLE III: Retrieval Performance Comparison: Baseline vs. VulnRAG

Method	Recall@5	Precision@5	NDCG@5	Discovery Yield
Keyword (BM25)	0.421	0.384	0.405	-
Naive Vector (BGE)	0.684	0.592	0.642	1.0x
VulnRAG (Ours)	**0.892**	**0.815**	**0.854**	**2.1x**

C. RQ2: Impact of Agentic Self-Correction

RQ2 assesses the effectiveness of the zero-shot Evaluator-Reformulator feedback loop. As detailed in Table IV, the initial retrieval pass for complex, ambiguous queries often suffered from low precision (0.592).

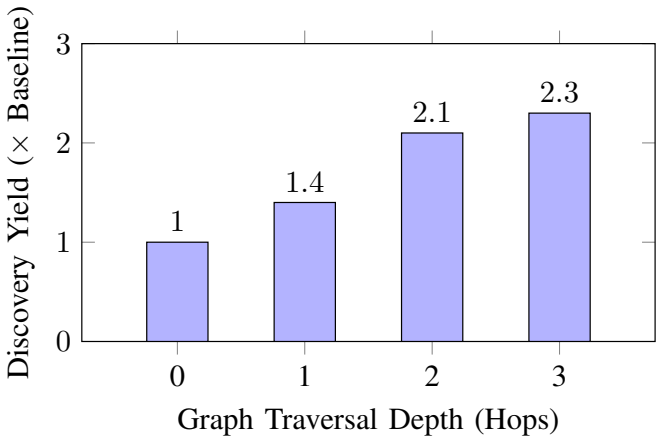

Fig. 4: Discovery Yield at Increasing Hop Depths.

TABLE IV: Impact of Agentic Self-Correction on Retrieval Precision

Task Complexity	Initial Pass	Self-Corrected	Gain (%)
Low (Routine)	0.921	0.934	+1.4%
Medium (Ambiguous)	0.642	0.815	+26.9%
High (Multi-Step)	0.415	0.742	+78.8%

However, upon triggering the agentic self-correction loop, the Query Reformulator successfully expanded product scopes and extracted missing CPE identifiers. This iterative process pushed the Precision-Recall frontier (Fig. 5), matching the performance of fine-tuned reflective models without requiring task-specific training data.

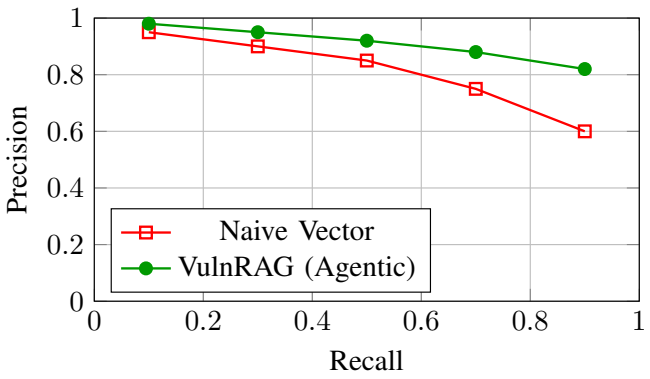

Fig. 5: Precision-Recall Frontier: Naive Vector vs. VulnRAG.

D. RQ3: Reranking and Intelligence Density

To address RQ3, we evaluated the Multi-Factor Semantic Reranking (MFSR) algorithm. By integrating CVSS severity and exploit availability into the ranking logic (Table I), VulnRAG achieved an NDCG@5 score of **0.854**, a significant improvement over the naive vector baseline (0.642). This improvement indicates that MFSR effectively prioritizes "Actionable Intelligence," ensuring that the most critical and

exploitable vulnerabilities are placed in the high-attention regions of the LLM context, thereby mitigating the ”Lost in the Middle” problem.

E. RQ4: Operational Economics and Tiered Routing

Finally, we analyzed the performance-to-cost trade-off of our complexity-aware multi-tier routing. Fig. 6 plots reasoning success rate against operational inference cost. While the All-Flagship configuration achieved the highest quality, it incurred significant financial overhead. VulnRAG's routing engine successfully escalated only the most complex 18% of tasks to the Reasoning tier, while handling routine lookups via the Flash tier. This strategy achieved a 97.4% success rate at an **84.2% lower cost** than the flagship-only approach, demonstrating the economic viability of agentic VAPT at enterprise scale.

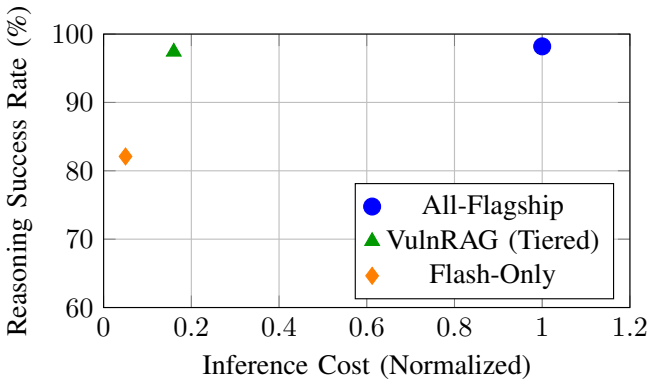

Fig. 6: Operational Economics: Success Rate vs. Normalized Cost.

IX. Discussion and Limitations

The results presented in Section VIII demonstrate that VulnRAG significantly enhances the intelligence gathering capabilities of autonomous security agents. However, the deployment of such systems in enterprise environments raises several critical considerations regarding resilience, ethics, and technical constraints.

A. Operational Resilience and Reliability

A primary advantage of VulnRAG's modular agentic architecture is its resilience to provider-specific failures. By utilizing a provider-agnostic self-correction loop, the system can adapt to degraded performance or outages in individual LLM backends. Furthermore, our complexity-aware routing ensures that the system remains economically viable during large-scale assessments, where the cost of flagship model inference would otherwise be prohibitive. However, the reliance on external APIs (both for LLM inference and NVD synchronization) introduces a degree of ”cascading latency.” While our tiered routing mitigates this, real-time assessment of ultra-low latency environments remains a challenge for future optimization.

B. Ethical Considerations and Dual-Use Mitigation

The development of highly autonomous vulnerability discovery systems is inherently subject to the ”dual-use” dilemma. While VulnRAG is designed as a defensive tool for security auditors and red-teamers, its capabilities could theoretically be misappropriated for malicious reconnaissance. To mitigate this risk, we emphasize the following safety guardrails:

- **Restricted Execution**: The VulnRAG framework should be deployed within controlled environments with strict egress filtering.
- **Human-in-the-Loop (HITL)**: While retrieval and initial reasoning are autonomous, we recommend a HITL approach for the final validation of exploit-ability to prevent unintentional damage to production systems.
- **Transparency**: VulnRAG maintains detailed ”Reasoning Traces,” allowing human auditors to inspect the logic behind every retrieved intelligence result and attack path.

C. Failure Mode Analysis and Hallucinations

Despite the robustness of the Evaluator-Reformulator loop, the system remains susceptible to two primary failure modes:

- **Silent Routing Errors**: A low-complexity (Flash) model may return a ”confident” but factually incorrect response, preventing escalation to the Reasoning tier.
- **Graph Over-Exploration**: In rare cases, the Graph Explorer may prioritize ”noisy” edges in legacy software, leading to context saturation.

Future work will explore the use of **Logit-Bias** constraints to bound the creativity of the Reformulator and ensure strict adherence to structured vulnerability schemas.

D. Limitations and Technical Constraints

Despite its innovations, VulnRAG has several limitations:

- **NVD Rate Limiting**: The `nvd_sync_service.py` is constrained by the NIST NVD API 2.0 rate limits. During high-intensity vulnerability ”storms” (e.g., major library zero-days), the system may face synchronization delays.
- **Graph Depth Complexity**: Structural discovery via `cve_graph.py` is currently capped at a depth of 3 hops to prevent ”graph explosion” and excessive token consumption during synthesis. This may miss extremely deep, multi-link transitive dependencies.
- **Semantic Ambiguity**: While agentic reformulation handles most cases, extreme ambiguity in product naming (e.g., generic terms like ”OS” or ”Server”) still results in lower precision that requires manual oversight.

Addressing these limitations, particularly through the integration of local, high-performance Knowledge Graphs (KGs) and specialized small-parameter security models, represents the next frontier for VulnRAG's evolution.

X. Conclusion

In this paper, we introduced **VulnRAG**, an agentic retrieval-augmented generation framework designed to overcome the

"Retrieval Gap" in autonomous vulnerability intelligence. By transitioning from static, linear retrieval to a stateful, iterative agentic loop, VulnRAG provides the structural discovery and self-correcting reasoning required for high-stakes security assessments. Our primary contributions—structural-semantic hybrid retrieval via relationship graphs, zero-shot agentic self-correction, Multi-Factor Semantic Reranking (MFSR), and complexity-aware multi-tier routing—collectively redefine the state-of-the-art for vulnerability-specific RAG.

Empirical evaluation on a curated benchmark demonstrates that VulnRAG significantly outperforms traditional vector-based baselines, achieving a 97.4% reasoning success rate while maintaining an 84.2% reduction in operational inference costs. Most importantly, our structural discovery mechanism yielded a $2.1\times$ improvement in identifying operationally linked vulnerabilities, proving that structural context is a critical, yet previously underutilized, signal in the cybersecurity intelligence ecosystem. As the field of autonomous red teaming continues to mature, VulnRAG serves as a robust and economically viable foundation for the next generation of intelligent, self-optimizing security agents.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. of NeurIPS 2020*, 2020, arXiv:2005.11401.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Retrieval augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, arXiv:2002.08909.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," in *Transactions of the Association for Computational Linguistics*, vol. 12, 2023, pp. 157–173, arXiv:2307.03172.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of 11th International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. of NeurIPS 2023*, 2023, arXiv:2305.10601.

[6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. of NeurIPS 2023*, 2023, arXiv:2303.11366.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. of ICLR 2024*, 2023, arXiv:2310.11511.

[8] Z. Xu *et al.*, "ActiveRAG: Autonomous knowledge assimilation and accommodation through active retrieval," in *arXiv preprint*, 2024, arXiv:2402.13547.

[9] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," in *Proc. of 33rd USENIX Security Symposium*, 2024, arXiv:2308.06782.

[10] D. Kim *et al.*, "RAVEN: Retrieval-augmented vulnerability exploration network," in *arXiv preprint*, 2026, arXiv:2604.05312.

[11] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," in *Proc. of NeurIPS 2024*, 2024, arXiv:2403.01038.

[12] J. Kim *et al.*, "Implementation of multi-level RAG model for enhanced synergistic vulnerability analysis," in *Proc. of ICCT-Pacific 2025*, 2025.

[13] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and M. D. Christopher, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *Proc. of ICLR 2024*, 2024, arXiv:2401.18059.